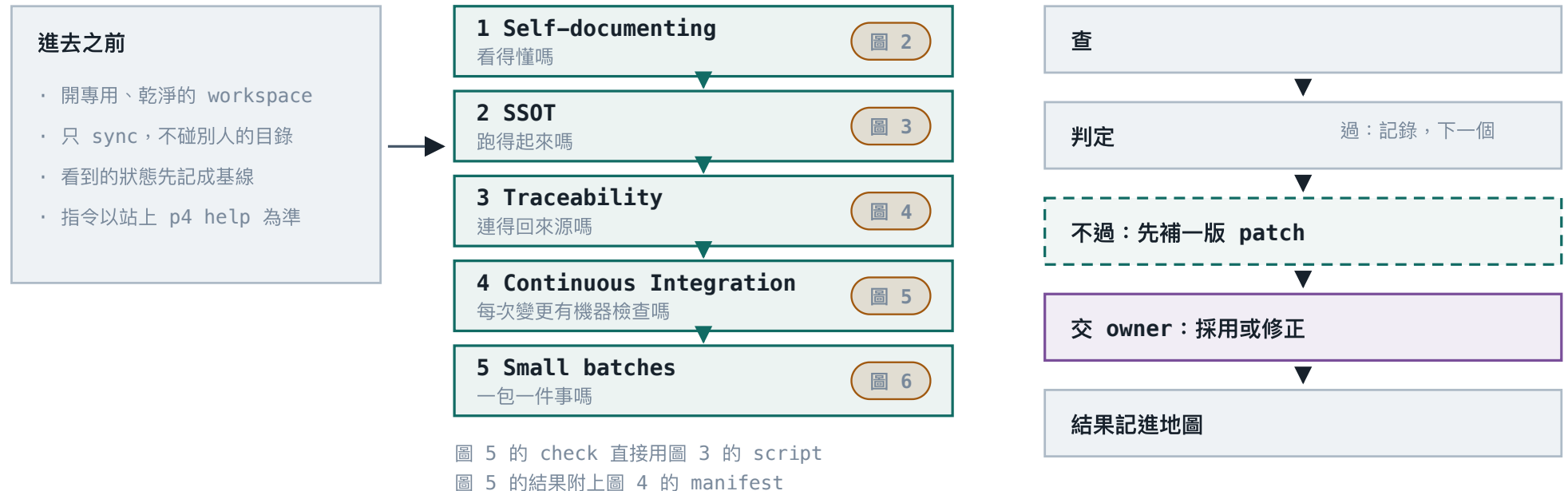


流程：五個檢查各驗一個原則，不過就先補一版給 owner 採用

這份文件回答：第一次進一個沒看過的 workspace，怎麼具體檢查它有沒有 CI/CD；檢查點不過時，自己先補什麼 ↓

五個檢查，照這個順序

每個檢查都走一遍



五個檢查各對應一個原則的檢驗；不過的時候，多數缺口靠通用常識就能先補一版。

owner 拿到的是可以直接採用的 patch，只有少數幾件事需要他回答。

Self-documenting：只讀 repo 說不出目錄用途，就先寫一版地圖

Self-documenting

檢驗：第一次來的人或 AI agent 只讀 repo，就說得出每個目錄的用途與相依

怎麼查

- 列頂層目錄，找 README、說明檔
- 找 filelist 與 run script
- 對照：哪些檔案沒被任何 filelist 引用
- 看最近的 CL：哪些目錄還有人在動

判定

- ✓ 每個頂層目錄說得出用途
- ✓ 誰引用誰畫得出來
- ✓ 怎麼跑有一份寫下來的

不過的徵兆

- ✗ rtl_old/ 與 rtl_new2/ 並存
- ✗ README 寫的是上一個專案
- ✗ 沒有任何檔案說怎麼跑

不過時，先補這些（自己做）

PROJECT_MAP.md 初稿

每個目錄一行用途，標依據：擷取 / 推測 / 猜的

相依圖

從 filelist 擷取誰引用誰

疑似未使用清單

沒被引用也沒人動的目錄，只標不刪

只需要問 owner

? 標「猜的」那幾行對不對

? 疑似未使用的目錄能不能標為棄用

看懂的依據要分開標：擷取的、推測的、猜的，owner 只需要看猜的那幾行。

沒被引用的目錄先標、不刪；刪不刪是 owner 的事。

SSOT：乾淨的 workspace 跑不起來，缺的先補成 script 進 depot

Single Source of Truth (SSOT)

檢驗：換一台乾淨的機器，只靠版控的內容做出同一個結果

怎麼查

- 專用 workspace 只 sync，不帶本機設定
- 照 repo 裡寫的方式編 top、跑一個最小的 sim
- 每個失敗分類：缺檔 / 缺工具 / 缺環境 / 缺 script
- 跟 owner 手上的結果比對

判定

- ✓ 編得過、跑得完
- ✓ 結果和 owner 手上的一致

不過的徵兆

- ✗ filelist 引用不存在的檔
- ✗ script 在 /home 或 /proj 底下
- ✗ 工具版本只在 .cshrc 裡

不過時，先補這些（自己做）

setup.sh

工具版本、環境變數寫死在裡面，進 depot

run_smoke.sh

從乾淨 workspace 編 top+跑一個 sim

缺檔清單

哪個 filelist 第幾行引用、誰最後動過

.p4ignore 提案

進了 depot 的產生檔列出來

只需要問 owner

? 缺的檔在誰的 workspace

? 產生檔要不要移出版控

跑不起來的每一個原因，都是一件散在 depot 外面的東西；補法是把它寫成 script 進 depot。

只有「缺的檔在誰那裡」這件事非問 owner 不可。

Traceability：交付物說不出來源，就補一份 manifest 跟著它走

Traceability

檢驗：隨便拿一份結果，說得出它的版本、工具、環境與步驟

怎麼查

- 拿最近的 label、release 包、regression 報告
- 問四件事：CL、工具版本、環境、怎麼跑
- 用內容指紋比對 release 包和 label 的檔案

判定

- ✓ 四件事都從紀錄答得出，用不著問人
- ✓ release 包和 label 對得上

不過的徵兆

- x label 只有檔案清單
- x 報告是一個會變的路徑
- x release 包手動打，README 靠回想

不過時，先補這些（自己做）

make_manifest.sh

寫出 CL、label、工具版本、環境、指令、結果摘要

manifest 跟著結果走

放進 run 目錄與 release 包，一起進 depot

舊交付物反查

指紋對得上就補 manifest，對不上標「無法追溯」

只需要問 owner

? 交付物的正式清單：給誰、含什麼

manifest 是結果和來源之間缺的那條線；讓產生結果的 script 順手寫出來，就不用靠人記。

過去的交付物追不回來的，標成基線，從現在開始追。

CI：submit 後沒有機器檢查，就先把 smoke check 排上去跑

Continuous Integration (CI)

檢驗：改壞的那一包進來時就被標出來，用不著等到整合

怎麼查

- 找 submit 觸發的檢查：p4 triggers、CI job
- 找定期 regression：結果放哪、誰看得到
- 翻最近一次壞掉：多久後、被誰發現

判定

- ✓ 每個 submit 都有機器跑過最小檢查
- ✓ 結果公開看得到，附 CL 號

不過的徵兆

- ✗ 只有人手動跑
- ✗ 有 nightly，結果只在跑的人信箱
- ✗ 壞掉由下游先發現

不過時，先補這些（自己做）

最小 check

拿圖 3 的 run_smoke.sh：編得過+一個 sim

排上去跑

先定時跑，再進到每個 CL 都跑

結果可見

寫到 repo 看得到的地方，附 CL 號與 manifest

只需要問 owner

? 觸發要裝在哪、誰有權限

? 算力與 license 的配額

? 壞了通知誰

最小的 check 就是圖 3 那個 script；先讓它定時跑、結果看得到，再談每個 CL 都跑。

裝觸發、配算力、通知誰，要 owner 點頭。

Small batches：歷史改不了，補的是說明模板與拆包的示範

Small batches

檢驗：隨便挑一包 submit，說得出它改了哪一件事

怎麼查

- p4 changes 看最近的 submit：幾個檔案、說明
- 挑幾包用 p4 describe 看：一包裡幾件事
- 看兩包之間隔多久

判定

- ✓ 一包一件事
- ✓ 說明看得懂改了什麼、為什麼

不過的徵兆

- ✗ 說明是 update、fix、sync
- ✗ 一包混 RTL、tb、script
- ✗ 一包是某人一整段期間的工作

不過時，先補這些（自己做）

CL 說明模板

改了什麼 / 為什麼 / 怎麼驗

pre-submit 清單

跑過 smoke、filelist 更新、一包一件事

拆包示範

拿一包進行中的工作，用 shelved CL 拆成幾包

只需要問 owner

? 團隊願不願意用模板

? 進行中的大包怎麼拆，當事人決定

submit 的歷史改不了，能補的是下一包的寫法：模板、清單、一個拆開的示範。

大包怎麼拆，只有當事人知道哪些改動是一起的。

分工：自己能補的直接交 patch，只有三類事才問 owner



patch 怎麼交



從頭到尾守住的



多數缺口靠通用常識就補得出第一版；owner 的工作從回答問題變成採用或修正 patch。

三類事只有 owner 知道；問的時候附上你已經查到的部分。